

Agent Skills で作業を設計する

Codex CLI は作業場所と安全境界を支える

uma-chan

2026-05-28



1. 何を分けるか

1.1. この資料でできるようにすること

この資料では Codex CLI の機能一覧ではなく、Agent Skills で作業の判断材料を再利用する話をします
持ち帰ってほしいこと

- 弱い依頼を「前提不足」として見られる
- 繰り返す手順を Agent Skills に寄せられる
- [SKILL.md](#) の入口と本文を分けて考えられる
- Codex CLI の作業場所と安全境界を決められる
- DONE を証跡で確認できる

Codex CLI は、Agent Skills を使う時の作業場所と安全境界を支える道具として扱う

1.2. 弱い依頼は前提不足

弱い依頼は、必要な判断材料がまだ外に出ていない依頼

足りないもの 起きること

前提	どの前提で作業するか推測される
作業手順	作業順序が毎回変わる
制約	やってはいけないことが漏れる
確認方法	確認せずに完了したことになる
証跡	何を根拠に完了したか残らない

短い依頼が悪いわけではない

短くても、必要な前提、手順、制約、確認、証跡が渡っていれば使える

1.3. 判断材料の置き場所

全部を 1 つのプロンプトや CLI オプションに詰め込まない

置き場所	入れるもの
作業依頼	今回の目的、対象、例外
チケット	背景、受け入れ条件、ユーザーへの影響
Agent Skills	繰り返す作業順、確認方法、報告形式
プロジェクト文書	リポジトリ固有の約束
Codex CLI	作業場所、追加の書込範囲、安全境界
チェック	機械的に確認できる結果

短い依頼は、情報を捨てることではない
情報を、毎回書かなくても読まれる場所へ移すこと

2. Agent Skills を設計する

2.1. 一次情報とカタログ

Agent Skills の構造は、まず公式仕様を見る

見る場所	位置づけ
https://agentskills.io/home	Agent Skills の標準
https://agentskills.io/specification	SKILL.md、必須項目、検証
https://github.com/openai/skills	Codex で使う Skill の配布元
<code>waza</code> 、 <code>gh skill</code>	手元で確認する道具

どれが標準で、どれが配布元で、どれが手元の道具かを分けておく

2.2. 小さな専門手順書

Agent Skills の 1 単位は、特定の作業領域に対する「使うタイミング」と「進め方」

```
skills/scoped-slide-work/SKILL.md
1 ---
2 name: scoped-slide-work
3 description: 発表資料を小さく修正し、描画確認と証跡報告まで行う。
4 ---
5
6 # 発表資料を限定修正する
7
8 ## 作業手順
9
10 1. 依頼、対象ファイル、制約、完了条件を読む。
11 2. 現在の資料と関連する参照を確認する。
12 3. 依頼範囲に収まる最小の修正をする。
13 4. 対象資料を描画確認し、Markdown のチェックを通す。
14 5. 変更ファイル、確認結果、残ったリスクを報告する。
```

これくらい小さい方が、いつ読むべきか判断しやすい

2.3. 入口と本文を分ける

Agent Skills は、最初から全文が読まれるとは限らない

項目	役割
name	安定した識別子
description	いつ読むべきか、いつ読まないかの判断材料
本文	作業時の手順、制約、確認、報告形式
references/	長い背景や例
scripts/	再利用する補助処理

本文が良くても、入口が弱いと選ばれにくい

2.4. 標準と作業しやすい型

標準で必須なのは `name` と `description`

それ以外は、この資料では作業しやすくするための型として足す

領域	位置づけ	置くもの
<code>name</code>	標準の入口	安定した識別子
<code>description</code>	標準の入口	使う場面と使わない場面
作業手順	足す型	作業開始から報告までの 3 から 5 手順
確認方法	足す型	最低 1 つの確認コマンド
報告形式	足す型	短い依頼でも返してほしい証跡
参考資料	足す型	長い背景や例へのリンク

長い背景説明は後から足す

2.5. 日本語と避けたいこと

Agent Skills の日本語本文は禁止されていない

判断	方針
name	小文字英数字とハイフンに寄せる
description	自動選択で読まれるため、短く具体的に書く
本文	実作業の手順は読みやすさを優先して日本語でもよい
references/	長い背景や例として日本語でもよい

評価用の依頼例 日本語の依頼を扱うなら日本語例も入れる

避けたいのは、何でも入った Skill、長い文書の丸ごとコピー、検証できない雰囲気ルール

3. Codex CLI で使う

3.1. Codex は実行面を設計する道具

公式資料から拾える Codex の話は、機能の羅列ではなく実行条件の整理に使える

公式資料で見えるもの	発表での意味
AGENTS.md	リポジトリの常時ルールを起動時の文脈に入れる
Skills	短い description で選ばせ、必要な手順だけ読ませる
Permissions / sandbox	書ける場所、ネットワーク、承認の境界を決める
CLI / MCP	作業場所、追加ディレクトリ、外部ツールへの入口を決める

Codex は「全部知っている相手」ではなく、文脈、手順、権限、道具をどう渡すかで働きが変わる実行環境として扱う

3.2. Codex CLI の役割

Codex CLI を起動する前に、まずこれだけ決める

観点	決めること	例
作業場所	どのリポジトリで起動するか	Web アプリ、API、ライブラリ
変更対象	どの範囲を触ってよいか	ログイン画面、注文 API、README
使う Skill	どの手順を呼び出すか	テスト、レビュー、リリースノート
触らないもの	同時進行の変更や生成物の	DB マイグレーション、認証設定、生成ファイル
完了条件	何が通れば終わりと言えるか	テスト、表示確認、 <code>git diff --check</code>

ここが曖昧だと、Agent Skills が良くても作業の境界がぼやける

3.3. 作業範囲の宣言

```
1 cd path/to/project
2 codex
3
4 # 別のディレクトリも編集する時だけ追加する
5 codex --add-dir ../related-repo
```

--add-dir は、読ませるだけの指定として扱わない

状況	判断
追加先を編集する	--add-dir を使う
追加先を読むだけ	パスと目的を依頼文に書く
どこを編集するか未確定	先に対象を絞る
本番データに近い場所	使わない

範囲を広げる前に、なぜ広げるかを書く

3.4. 依頼文の型

- 1 目的
- 2 ログイン画面のエラー表示を分かりやすくしてください。
- 3
- 4 対象
- 5 `frontend/src/pages/login.tsx`
- 6
- 7 使う Agent Skills
- 8 React コンポーネント修正とテストの手順を優先する。
- 9
- 10 触らないもの
- 11 認証設定、API、生成ファイル
- 12
- 13 完了条件
- 14 対象のテストと lint が通ること。
- 15 DONE に変更内容と証跡を書くこと。

依頼の意図、作業範囲、止まる条件を同時に伝える

3.5. 設定はガードレール

Codex の設定では、作業を始める前の固定条件を決める

観点	設定やフックで固定すること
Skills	Skills 機能を有効にし、必要な Skill を読み込ませる
作業場所	信頼するプロジェクトだけを明示する
サンドボックス	読み取り専用、作業場所だけ書込、制限なしを選ぶ
コマンドフック	<code>rm</code> 、 <code>sudo</code> 、 <code>git push</code> 、 <code>git reset</code> などを止める
入力フック	共通ルールを起動時の文脈に入れる
書込範囲	<code>--add-dir</code> と依頼文で明示する

手順は Agent Skills に置き、Codex 設定には使える範囲と止める操作を置く

4. 使って確認する

4.1. 手元で確認してから共有する

Agent Skills は一度で完成しない

```
1 waza --no-update-check check skills/<name> --format json
2 gh skill publish --dry-run
```

確認するもの 減らせる事故	
waza	必須項目の漏れ、足りない説明、構造の崩れ
gh skill	共有や公開の前に、配布できない形に気づく
レビュー	何を確認済みか、DONE の証跡で追える

仕様は一次情報で見る。手元の確認で、共有前の手戻りを減らす

4.2. DONE は証跡で見る

DONE は終了宣言だけではなく、確認するための入口

```
1 DONE: Agent Skills 中心の発表資料へ整理しました。  
2 Task artifact: <task-artifact-path>  
3 Original checklist: PASS  
4 Remaining blockers: none  
5  
6 Evidence  
7 - 対象スライドを Quarto で描画確認しました。  
8 - Markdown チェックが通りました。  
9 - git diff --check が通りました。
```

人間が見る順番は、変更範囲、確認結果、タスク成果物、残ったリスク

DONE の文章だけで判断しない

4.3. 今日のまとめ

テーマ	要点
弱い依頼	前提、手順、確認方法、証跡が足りない状態
Agent Skills	運用知識を再利用可能にするパッケージ
Codex CLI	作業場所、書込範囲、止める操作を決める道具
確認ツール	waza で構造、gh skill で配布できる形かを見る
DONE	証跡を見て判断する

構造は標準で確認し、ツールは手元の確認に使う